

Deploy

مشكلة ← لما كنت بدخل على صفحة الـ Register
بيظهر Error ← HTTP Status 403 - Forbidden

يعني أي HTTP 403 ← لما ال Browser
يحمل req للسيرفر والسيرفر يرجع HTTP Status Code
مثلا ← 200 OK يعني الطلب نجح

أما لو رجع 403 Forbidden
معناها ال Server فهم ال request لكنه رفض
السماح بتنفيذ

اللي حصل عندي في المشروع
ال app1 عنده 2 Pod وال Service كان عنده EndPoint
لكل Pod
10.0.13.216:8080
10.0.11.198:8080

وكان ال Session Affinity: None
يعني ال Kubernetes Service مني بيضمن ان Requests
بتاعة نفس ال Client تروح لنفس ال Pod

طوبى يعني أي Session ← في VProfile تطبيق Tomcat/Jetty
قديم وبيعتمد على Tomcat Session
لما تدخل الصفحة Get/registration ال App بيحمل Session
وتسبب مع ال req

Deploy

1 مشكلة

طبيب في العلاقة ال CSRF

ال VProfile يستخدم Spring Security CSRF Protection

ال Spring Security يحاول يتأكد ان POST request
اللي جاي لـ APP اتعمل من الفورم الحقيقي بتاع ال APP
من req مزود من موقع ثاني

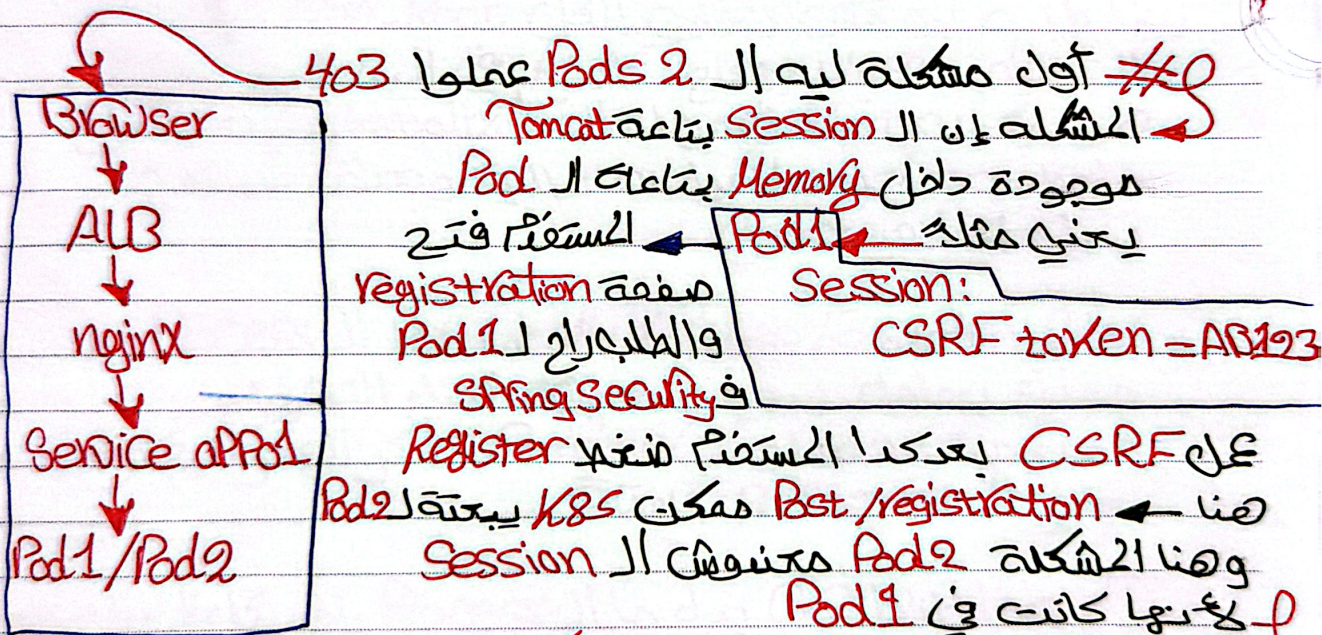
ال CSRF Token هو Token امن بيستخدمه
ال APP عشان يتأكد ان ال POST request جاي
بطريقة صحيحة ومن session صحيح

في ملف Registration.jsp دي الصفحة اللي فيها reg form
ال user بيستخدمها بكن المشكلة ان ال form عندي
من بيبيته CSRF



Troubleshooting - registration - 403

- 1) المشكلة بعد ما عملت deploy لا VRafile على ال EKS ال APP كان شغال لكن لما كنت بعمل Registration بيرجع ال 403 مش بسبب واعد ال Session موجود في ال Memory وال Pod عندهم 2 ال 403 بسبب ال CSRF
- 2) ال Hibernate Auto يحتاج hibernate sequence مش موجوده 500
- 3) اختبار Curl نفسه كان بيطلع على ال Pods 2 ال 403 تاني



ال Spring Security يقول ان اعندي CSRF token جاي من المستخدم لكن ال Session اللي المفروض يكون فيها ال Token مش موجوده فيطالعني = 403 Forbidden

ال Tomcat بيخزن ال HTTP session داخل ال Pod و Spring Security بيخزن CSRF Session داخل ال Session

Troubleshooting - Registration - 403

مشكلة الـ **Sticky Session** حلت جزء من المشكلة
لأن الـ **User** لما يبيعت **Req** بيرجعة
لنفس الـ **Pod** مبيخترش الـ **Nginx Pod**
لكن كان في **Trap** مهم

هنا افنا حلينا مشكلة الـ **Req** للـ **Nginx**
لكن مش بيضمن برضوان الـ **Req** يتجيب
لنفس الـ **app Pod** ساعتها الـ **Session** ضاعت برضو

بالتالي ضفنا **Session Affinity**
بدل ما كل **Req** يتوزع عشوائي الـ **Kubernetes Service**
يحاول يفضل يبعث **Req** من نفس الـ **Client IP**
لنفس الـ **Backend Pod**

لكن حصلت المفاجأة بعد ما عملت **Fix**
للـ **Session / CSRF** ظهر **500 Internal Server Error**

[2] مشكلة الـ **Hibernate**

ظهر في الـ **Response** ← **java.sql.SQLException: Table 'accounts.hibernate_sequence'**

بس محتاجين نفهم الأول يعني أي **Hibernate**
هو أداة بتدخل برنامج **Java** يتعامل مع **DB**
من غير ما تكتب كل أوامر **SQL** بنفسك

يعني مثلاً عن طريق **Java object** ← **Account** في الـ **DB**
مكتوب **accounts** يقوم الـ **Hibernate** بعمل الربط بينهم
يعني بدل ما أكتب **select * from accounts, insert into**
الـ **Hibernate** ممكن هو اللي يولد الـ **SQL** وينفذ

⑧ Troubleshooting - Registration - 403

نرجع للـ `response` اللي ظهر لنا

مناها ان `Hibernate` حاول يتعامل مع جدول
اسمته `hibernate_seq` جوا الـ `DB` اسمها `accounts`
لكن الجدول مش موجود

طبعه ليه `Hibernate` ليه محتاج `hibernate_sequence`

عندي `user` فيه `ID` الجزء المهم `GenerationType.AUTO`
يعني بقوله يا `Hibernate` افكار أنت الطريقة المناسبة
لتوليد قيمة الـ `id`